



3.5 自下而上语法分析

3.5.2 LR分析

1. LR分析与LR文法

2. LR(0)分析表的构造

2.1 LR(0)项目和LR(0)项目集规范族

2.2 LR(0)项目集规范族的构造

2.3 有效项目

2.4 LR(0)分析表的构造

3. SLR(1)分析表的构造（教材3.3.8节）

4. 非SLR(1)文法

5. 基于LR分析的语法分析器生成器简介



3.5 自下而上语法分析

2.1 LR(0)项目和LR(0)项目集规范族

思路：LR(0)分析表的构造：(1)构造一个可以识别文法G中所有活前缀的DFA；(2)根据DFA构造LR(0)分析表。

① 活前缀与LR(0)项目

定义(活前缀) 给定文法G，如果在符号序列 α 的右边增添零个或多个终结符之后，能够形成一个右句型并且 α 不含**该句型**句柄之后的任何符号，则称 α 为文法G的活前缀。

两个要素：右句型的前缀；不含句柄之后的任何符号。在LR分析的任何时刻，栈里的文法符号序列都是活前缀，如果输入串语法结构正确，则把输入串的剩余部分匹配于其后应形成右句型。因此，**只要保证已扫描过的输入序列可以归约为一个活前缀，则分析到目前为止没有错误。**

[例]举例说明以下文法的活前缀

G: $S \rightarrow aABe \mid aAbBec$ $A \rightarrow b \mid bA$ $B \rightarrow d$

活前缀：a, aA, aAB, aABe, aAb, aAbBec, aAbd, aAd,
ab, abb, abbb, ab^+



步骤	已扫描的前缀	栈(右端栈顶)	剩余输入	动作	右句型
(1)		#	abbcede#	移进	abbcede
(2)	a	#a	bbcede#	移进	abbcede
(3)	ab	#ab	bcde#	按 $A \rightarrow b$ 归约	abbcede
(4)	ab	#aA	bcde#	移进	aAbcede
(5)	abb	#aAb	cde#	移进	aAbcede
(6)	abbc	#aAbc	de#	按 $A \rightarrow Abc$ 归约	aAbcede
(7)	abbc	#aA	de#	移进	aAde
(8)	abbcd	#aAd	e#	按 $B \rightarrow d$ 归约	aAde
(9)	abbcd	#aAB	e#	移进	aABe
(10)	abbcede	#aABe	#	按 $S \rightarrow aABe$ 归约	aABe
(11)	abbcede	#S	#	接受	S

在输入序列语法结构正确的情况下，在语法分析的任意时刻，

- (1) 栈中的符号串+剩余输入串组成一个右句型；
- (2) 将已扫描的前缀部分归约为栈中的文法符号序列；
- (3) 从(1)~(11)，右句型的变化过程是abbcede最右推导的逆过程
 $abbcede \leq aAbcede \leq aAde \leq aABe \leq S$

3.5 自下而上语法分析

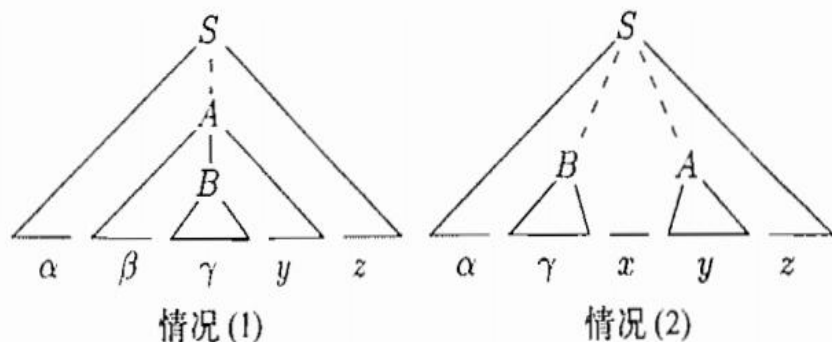
注意：

1. 句柄总是在栈顶形成（最左归约）：
语法分析器在进行一次归约之后，
都必须接着移进零个或多个符号才
能在栈顶找到下一个句柄。

2. 栈中总是含有一个右句型的前缀且
该前缀不包含句柄之后的符号——活前缀。

LR分析是如何解决这两个问题的？

- 1、如何判断栈顶是否形成句柄？
- 2、当句柄形成时，如何选择产生式？



[例3.27] 用移进-归约方法分析abbcde：

abbcde ≤ aAbcde ≤ aAde ≤ aABe ≤ S

栈	剩余输入	改变格局的动作
#	abbcde#	移进
#a	bbcd#	移进
#ab	bcde#	归约，(2) $A \rightarrow b$
#aA	bcde#	移进
#aAb	cde#	移进
#aAbc	de#	归约，(3) $A \rightarrow Abc$
#aA	de#	移进
#aAd	e#	归约，(4) $B \rightarrow d$
#aAB	e#	移进
#aABe	#	归约，(1) $S \rightarrow aABe$
#S	#	接受

考虑任意最右推导中的两个连续步骤，假设 $x, y, z \in T^*$, $\alpha, \beta, \gamma \in (N \cup T)^*$, $A, B \in N$

情况(1) $S \Rightarrow^* \alpha Az \Rightarrow^* \alpha \beta B \gamma z \Rightarrow^* \alpha \beta \gamma \gamma z \Rightarrow^* \alpha \beta x_3 \gamma z \Rightarrow^* \alpha x_2 x_3 \gamma z \Rightarrow^* x_1 x_2 x_3 \gamma z$

情况(2) $S \Rightarrow^* \alpha B x A z \Rightarrow^* \alpha B x \gamma z \Rightarrow^* \alpha \gamma x \gamma z \Rightarrow^* \alpha x_2 x \gamma z \Rightarrow^* x_1 x_2 x \gamma z$



3.5 自下而上语法分析

2.1 LR(0)项目和LR(0)项目集规范族

思路：LR(0)分析表的构造：(1)构造一个可以识别文法G中所有活前缀的DFA；(2)根据DFA构造LR(0)分析表。

① 活前缀与LR(0)项目

定义(活前缀) 给定文法G，如果在符号序列 α 的右边增添零个或多个终结符之后，能够形成一个右句型并且 α 不含该句型句柄之后的任何符号，则称 α 为文法G的活前缀。

两个要素：右句型的前缀；不含句柄之后的任何符号。在LR分析的任何时刻，栈里的文法符号序列都是活前缀，如果输入串语法结构正确，则把输入串的剩余部分匹配于其后应形成右句型。因此，**只要保证已扫描过的输入序列可以归约为一个活前缀，则分析到目前为止没有错误。**

构造LR(0)分析器的关键：为文法G构造一个识别它的所有活前缀的DFA。

步骤：NFA $\rightarrow$ DFA

问题：识别活前缀的NFA是什么？从NFA的初态到终态的路径上的标记就是活前缀。



3.5 自下而上语法分析

状态转换图： $E \rightarrow E + T$ $T \rightarrow T * F$



1. 识别活前缀的NFA：从初态到终态的路径上的标记是活前缀。
2. NFA的每个状态都应能够记录从初态到当前状态，已经看到了哪个产生式右部的多少内容。只有这样才能确定：如果分析器处于当前状态，则下一步是该移进还是归约，以及归约时用到的产生式。
3. 在产生式的右部加入一个点“.”，用它在右部的的位置表示一个NFA的状态。即，用一个LR(0)项目表示一个NFA状态。

定义3.17 一个**LR(0)项目**(简称项目)是这样产生式，在它右部的某个位置有一个点“.”。对于 $A \rightarrow \varepsilon$ ，它仅有一个项目 $A \rightarrow \cdot$ 。



3.5 自下而上语法分析

[例] 产生式 $A \rightarrow XYZ$ 有以下四个LR(0)项目：

$A \rightarrow \cdot XYZ$ $A \rightarrow X \cdot YZ$ $A \rightarrow XY \cdot Z$ $A \rightarrow XYZ \cdot$

LR(0)项目的意义：

一个项目指明了在分析过程中的某个时刻看到的产生式的多大一部分。

- $A \rightarrow \cdot XYZ$ ，说明希望从后面输入串中看到可以从XYZ推出的符号串，需要移进；
- $A \rightarrow X \cdot YZ$ ，说明已经从输入串中看到可以从X推出的符号串，希望进一步看到可以从YZ推出的符号串，需要继续移进。
- $A \rightarrow XYZ \cdot$ ，说明当前栈顶已经形成句柄，可以归约。



为文法G构造LR(0)分析表 $\Leftarrow$ 构造识别G的所有活前缀的DFA $\Leftarrow$ 构造识别G的所有活前缀的NFA $\Leftarrow$ 文法G的LR(0)项目 $\Leftarrow$ 文法G



3.5 自下而上语法分析

[例] 文法G3.14如下：（拓广之后的文法）

$S' \rightarrow E$ $E \rightarrow aA \mid bB$

$A \rightarrow cA \mid d$ $B \rightarrow cB \mid d$

由文法的LR(0)项目构造识别活前缀的NFA的方法：

- (1) 每个LR(0)项目都是NFA的一个状态；
- (2) Σ ：文法符号和 ϵ ；
- (3) 初态：左边为拓广文法开始符号且圆点在最左端的项目；
- (4) **终态：NFA的所有状态；**
- (5) 状态转换关系move：

若状态i和j出自同一个产生式，且状态j的圆点只落后状态i的圆点一个位置，例如，

状态i为 $X \rightarrow X_1 \dots X_{i-1} \cdot X_i \dots X_n$ ，状态j为 $X \rightarrow X_1 \dots X_i \cdot X_{i+1} \dots X_n$ ，
就从状态i画一条标记为 X_i 的边到状态j；

若状态i的圆点之后的那个符号为非终结符，如i为 $X \rightarrow \alpha \cdot A \beta$ ，A为非终结符，则从状态i画 ϵ 边到所有 $A \rightarrow \cdot \gamma$ 的状态。

该文法的项目有18个：

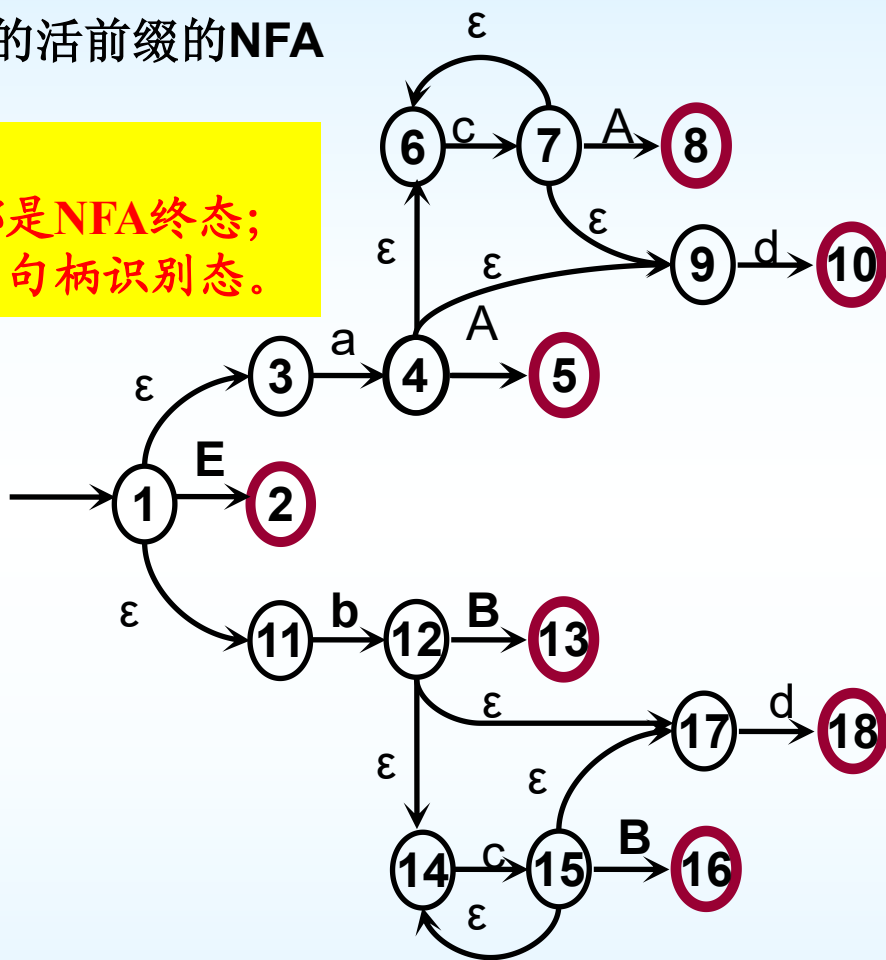
1. $S' \rightarrow \cdot E$
2. $S' \rightarrow E \cdot$
3. $E \rightarrow \cdot aA$
4. $E \rightarrow a \cdot A$
5. $E \rightarrow aA \cdot$
6. $A \rightarrow \cdot cA$
7. $A \rightarrow c \cdot A$
8. $A \rightarrow cA \cdot$
9. $A \rightarrow \cdot d$
10. $A \rightarrow d \cdot$
11. $E \rightarrow \cdot bB$
12. $E \rightarrow b \cdot B$
13. $E \rightarrow bB \cdot$
14. $B \rightarrow \cdot cB$
15. $B \rightarrow c \cdot B$
16. $B \rightarrow cB \cdot$
17. $B \rightarrow \cdot d$
18. $B \rightarrow d \cdot$



3.5 自下而上语法分析

识别文法G的活前缀的NFA

注意：
所有状态都是NFA终态；
红圈状态为句柄识别态。



该文法的项目有18个：

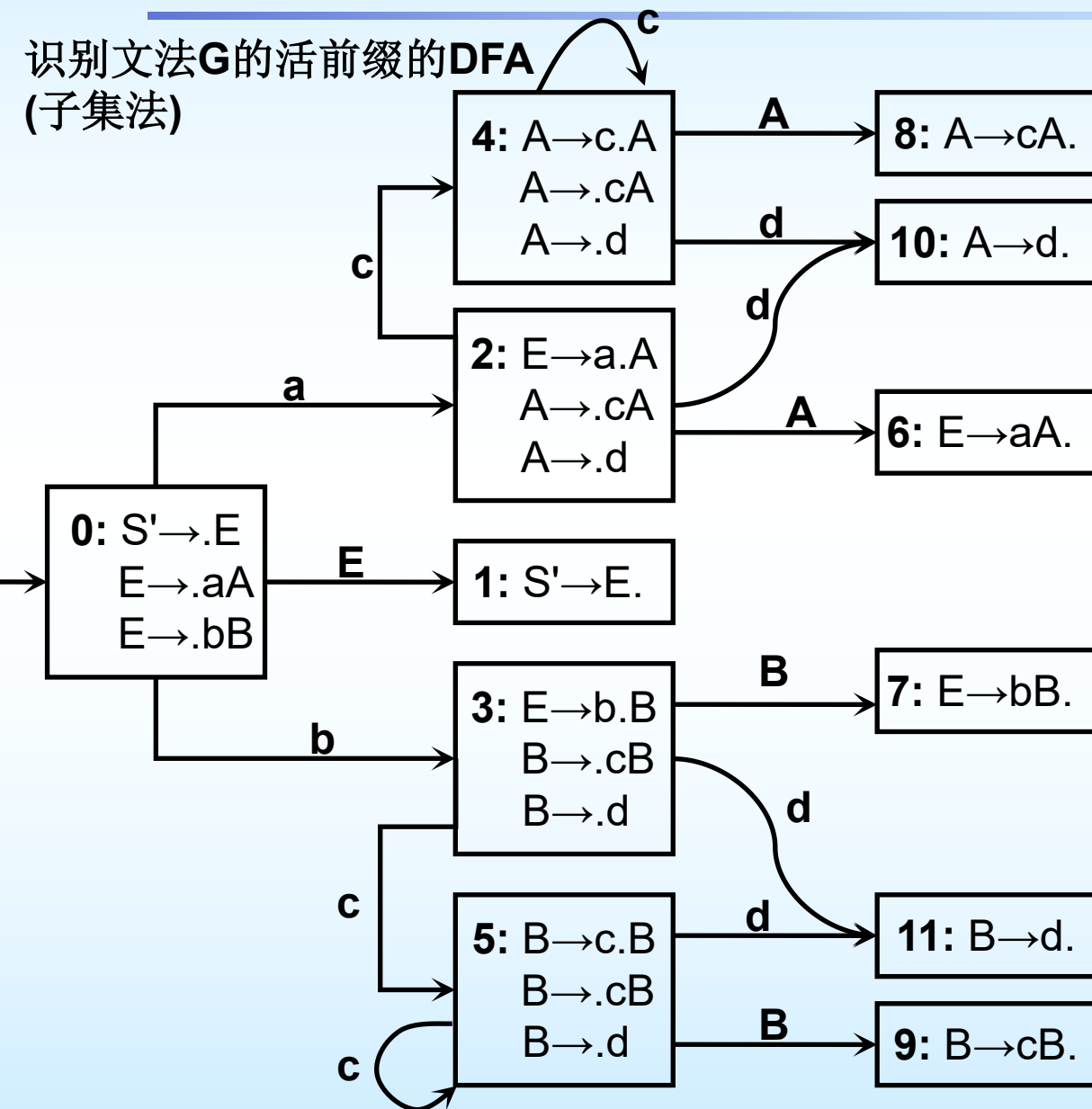
1. $S' \rightarrow .E$
2. $S' \rightarrow E.$
3. $E \rightarrow .aA$
4. $E \rightarrow a.A$
5. $E \rightarrow aA.$
6. $A \rightarrow .cA$
7. $A \rightarrow c.A$
8. $A \rightarrow cA.$
9. $A \rightarrow .d$
10. $A \rightarrow d.$
11. $E \rightarrow .bB$
12. $E \rightarrow b.B$
13. $E \rightarrow bB.$
14. $B \rightarrow .cB$
15. $B \rightarrow c.B$
16. $B \rightarrow cB.$
17. $B \rightarrow .d$
18. $B \rightarrow d.$

(1)若状态i和j出自同一个产生式，且状态j的圆点只落后状态i的圆点一个位置，例如，状态i为 $X \rightarrow X1 \dots Xi-1 . Xi \dots Xn$ ，状态j为 $X \rightarrow X1 \dots Xi . Xi+1 \dots Xn$ ，就从状态i画一条标记为 Xi 的边到状态j；(2)若状态i的圆点之后的那个符号为非终结符，如i为 9 $X \rightarrow a.A\beta$ ，A为非终结符，则从状态i画 ϵ 边到所有 $A \rightarrow .\gamma$ 的状态。



DFA的每个状态是一个LR(0)项目集 析

识别文法G的活前缀的DFA
(子集法)



该文法的项目有18个:

1. $S' \rightarrow \cdot E$
2. $S' \rightarrow E \cdot$
3. $E \rightarrow \cdot aA$
4. $E \rightarrow a \cdot A$
5. $E \rightarrow aA \cdot$
6. $A \rightarrow \cdot cA$
7. $A \rightarrow c \cdot A$
8. $A \rightarrow cA \cdot$
9. $A \rightarrow \cdot d$
10. $A \rightarrow d \cdot$
11. $E \rightarrow \cdot bB$
12. $E \rightarrow b \cdot B$
13. $E \rightarrow bB \cdot$
14. $B \rightarrow \cdot cB$
15. $B \rightarrow c \cdot B$
16. $B \rightarrow cB \cdot$
17. $B \rightarrow \cdot d$
18. $B \rightarrow d \cdot$



3.5 自下而上语法分析

由LR(0)项目直接构造识别活前缀的DFA

识别活前缀的DFA的状态是一个LR(0)项目集。

LR(0)项目集规范族：构成识别文法活前缀的**DFA的项目集(状态)的全体**称为这个文法的LR(0)项目集规范族。

这个规范族提供了建立一类LR(0)和SLR分析器的基础。

凡圆点在最右端的项目，如 $A \rightarrow \alpha \cdot$ ，称为“归约项目”；

对拓广文法开始符号 S' 的归约项目，如 $S' \rightarrow \alpha \cdot$ ，称为“接受项目”；

形如 $A \rightarrow \alpha \cdot a\beta$ 的项目，称为“移进项目”，其中 a 为终结符；

形如 $A \rightarrow \alpha \cdot B\beta$ 的项目，称为“待约项目”，其中 B 为非终结符；



3.5 自下而上语法分析

2.2 LR(0)项目集规范族的构造

a) 拓广文法 G' (不要忘记!!)

$$G' = G \cup \{S' \rightarrow S\}$$

既可以保证LR分析器有唯一“acc”状态，又能够将S识别为活前缀(保证S最终出现在栈中)

其中： $S' \rightarrow \cdot S$ 是识别S的初态， $S' \rightarrow S \cdot$ 是识别S的终态。目的是使最终构造的DFA状态集中具有唯一的“接受”状态，即唯一的指示语法分析成功结束的状态。

b) NFA(项目) $\rightarrow$ DFA(项目集)

词法分析器—“子集法”：

1. ϵ _闭包(I): 从状态集I仅经 ϵ 转移能到达的状态全体
2. $\text{smove}(I, a)$: 所有从I经字符a能直接到达的状态全体

类似的两个过程：

1. CLOSURE(I): 从项目集I不经任何文法符号到达的项目全体
2. GO(I, X): 所有从I经文法符号X能到达的项目全体(注：GO和smove功能不对等，GO含有闭包计算)



3.5 自下而上语法分析

2.2 LR(0)项目集规范族的构造

CLOSURE(I)的计算方法:

假设I是文法G'的任一项目集, 则

- (1) I的任何项目都属于CLOSURE(I);
- (2) 若 $A \rightarrow \alpha.B\beta$ 属于CLOSURE(I), 那么, 对任何关于B的产生式 $B \rightarrow \gamma$, 项目 $B \rightarrow \cdot\gamma$ 也属于CLOSURE(I);
- (3) 重复执行上述两步骤直至CLOSURE(I)不再增大为止。

[例] 对于文法G3.14, 假若 $I = \{S' \rightarrow \cdot E\}$, 则CLOSURE(I)所含项目为

$S' \rightarrow \cdot E$

$E \rightarrow \cdot aA$

$E \rightarrow \cdot bB$

G 3.14

$S' \rightarrow E$

$E \rightarrow aA \mid bB$

$A \rightarrow cA \mid d$

$B \rightarrow cB \mid d$



3.5 自下而上语法分析

CLOSURE(I)的计算方法:

假设I是文法G'的任一项目集, 则

- (1) I的任何项目都属于CLOSURE(I);
- (2) 若 $A \rightarrow \alpha.B\beta$ 属于CLOSURE(I), 那么, 对任何关于B的产生式 $B \rightarrow \gamma$, 项目 $B \rightarrow \cdot\gamma$ 也属于CLOSURE(I);
- (3) 重复执行上述两步骤直至CLOSURE(I)不再增大为止。

function CLOSURE(I) is

begin J := I;

for J中每个项目 $[A \rightarrow \alpha.B\beta]$ 和G中每个产生式 $B \rightarrow \gamma$

loop if $B \rightarrow \cdot\gamma$ 不在J中 then 加入 $[B \rightarrow \cdot\gamma]$ 到J; end if;

exit when 再没有项目可以被加入到J中;

end loop;

return(J);

end closure;



3.5 自下而上语法分析

2.2 LR(0)项目集规范族的构造

状态转换函数GO:

GO(I, X)的第一个变元I是一个项目集，第二个变元X是一个文法符号。**函数值GO(I, X)定义为**

$$GO(I, X) = \text{CLOSURE}(J)$$

其中， $J = \{\text{任何形如 } A \rightarrow \alpha X \beta \text{ 的项目} \mid A \rightarrow \alpha.X\beta \text{ 属于 } I\}$

[例]若 $I = \{S' \rightarrow .E, E \rightarrow .aA, E \rightarrow .bB\}$ ，则

$$GO(I, a) = \{E \rightarrow a.A, A \rightarrow .cA, A \rightarrow .d\} = \text{CLOSURE}(J)$$

其中 $J = \{E \rightarrow a.A\}$ 。



3.5 自下而上语法分析

2.2 LR(0)项目集规范族的构造

文法G的拓广文法G'的LR(0)项目集规范族的构造算法:

输入: 文法G'的所有LR(0)项目

输出: 文法G'的LR(0)项目集规范族, 存放在C中

```
procedure itemsets(G')
```

```
begin
```

```
  C := {CLOSURE({S' → .S})};
```

```
  repeat
```

```
    for C中的每个项目集I和G'的每个文法符号X do
```

```
      if GO(I, X)非空且不属于C then
```

```
        将GO(I, X)放入C中
```

```
      until C不再增大
```

```
end
```



3.5 自下而上语法分析

[例]文法 G' : $E' \rightarrow E$ $E \rightarrow E-T|T$ $T \rightarrow T*F|F$ $F \rightarrow -F|id$

计算CLOSURE($\{E' \rightarrow \cdot E\}$)

所有LR(0)项目:

$E' \rightarrow \cdot E$ $E' \rightarrow E \cdot$

$E' \rightarrow \cdot E$

$E \rightarrow \cdot E-T$ $E \rightarrow E \cdot -T$ $E \rightarrow E \cdot T$ $E \rightarrow E-T \cdot$

$E \rightarrow \cdot E-T$

$E \rightarrow \cdot T$ $E \rightarrow T \cdot$

$E \rightarrow \cdot T$

$T \rightarrow \cdot T*F$ $T \rightarrow T \cdot *F$ $T \rightarrow T \cdot F$ $T \rightarrow T*F \cdot$

$T \rightarrow \cdot T*F$

$T \rightarrow \cdot F$ $T \rightarrow F \cdot$

$T \rightarrow \cdot F$

$F \rightarrow \cdot -F$ $F \rightarrow - \cdot F$ $F \rightarrow -F \cdot$

$F \rightarrow \cdot -F$

$F \rightarrow \cdot id$ $F \rightarrow id \cdot$

$F \rightarrow \cdot id$

计算LR(0)项目集规范族时, 不需要把文法的所有LR(0)项目一一列出。

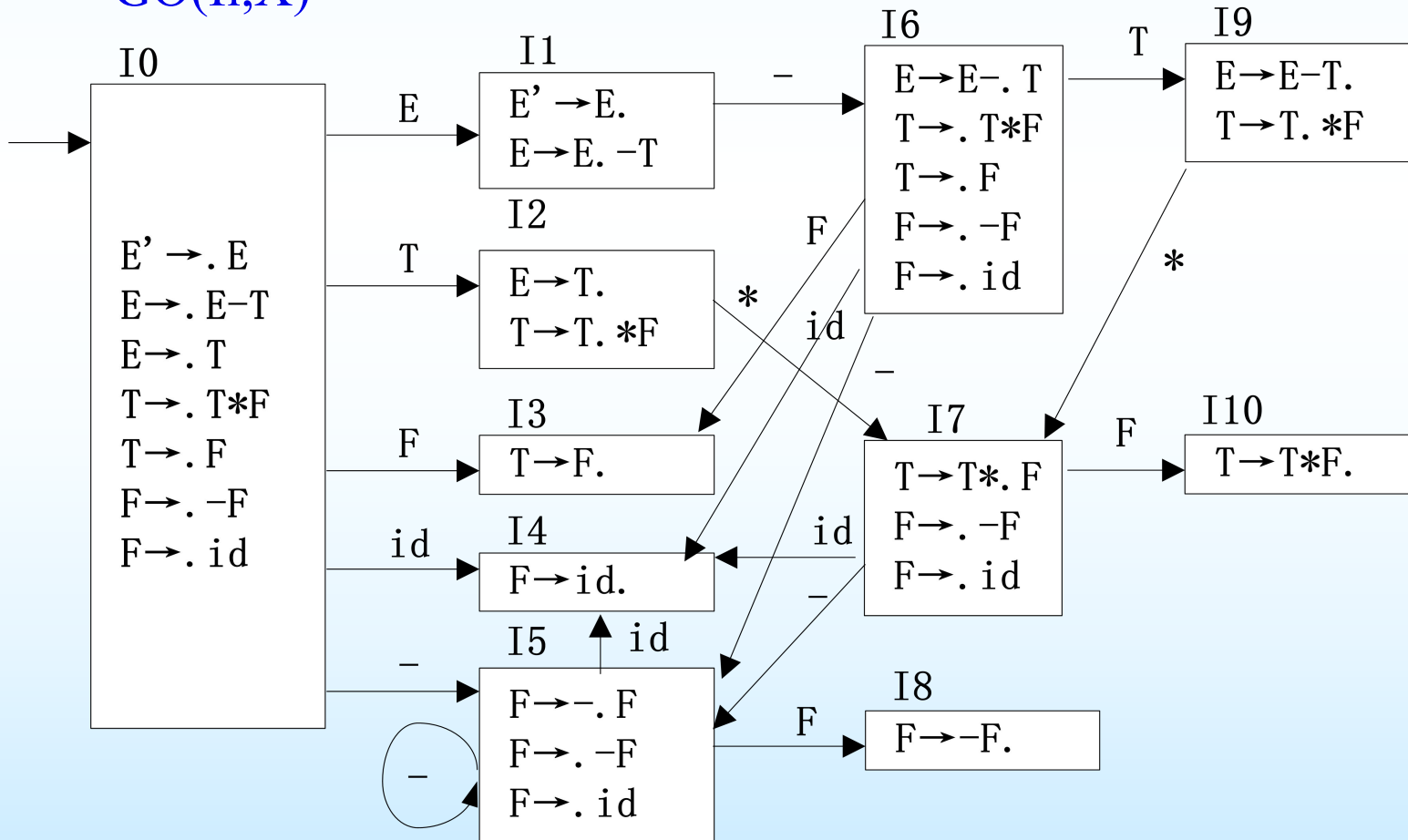


3.5 自下而上语法分析

构造DFA: 文法 G' : $E' \rightarrow E$ $E \rightarrow E-T|T$ $T \rightarrow T*F|F$ $F \rightarrow -F|id$

① 计算DFA的初态, $I_0 = \text{CLOSURE}(\{E' \rightarrow \cdot E\})$

② 计算所有状态的所有状态转移, 即考察每个未标记状态 I_i 的 $\text{GO}(I_i, X)$





3.5 自下而上语法分析

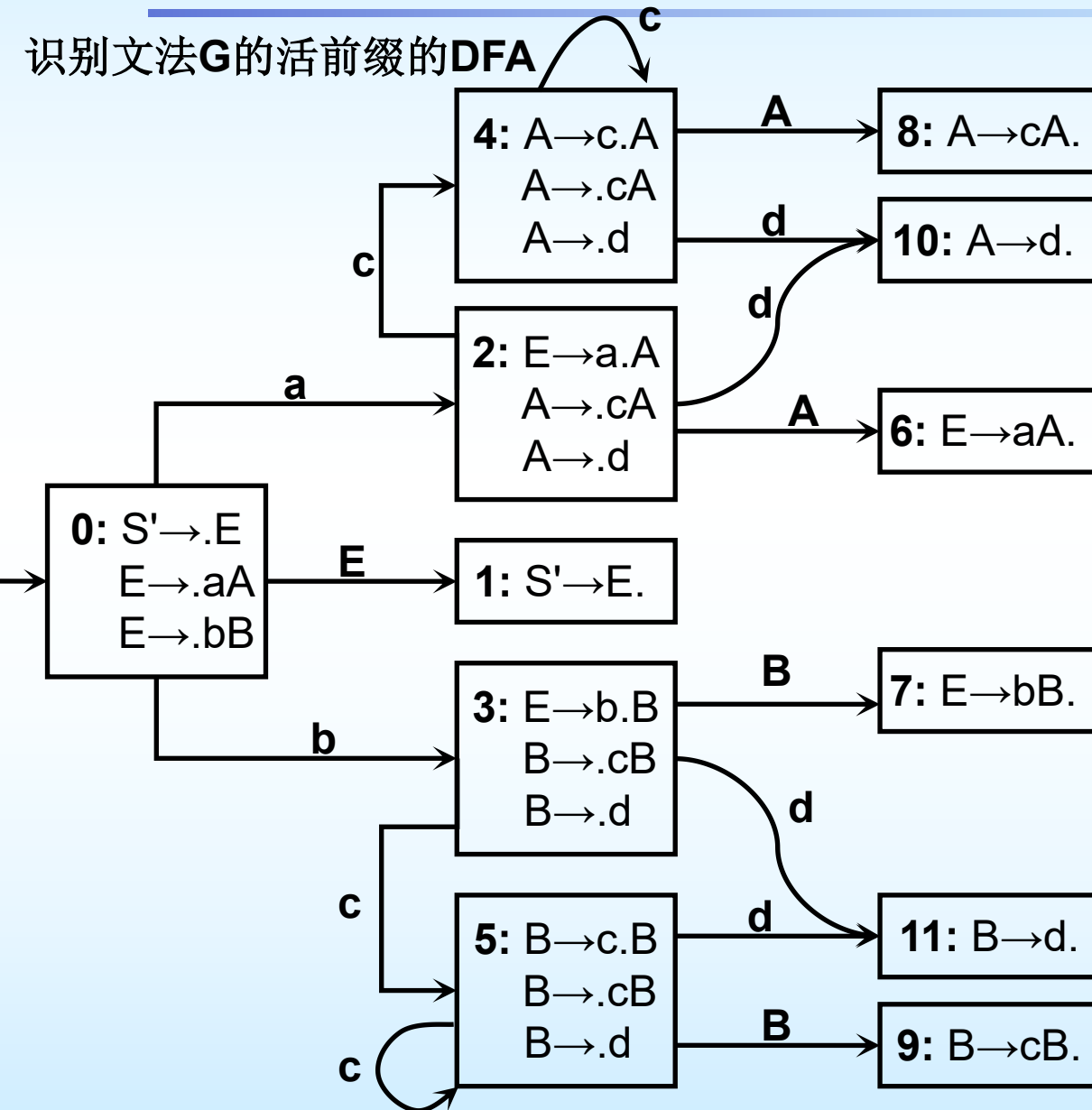
	id	-	*	#	E	T	F
0	s4	s5			1	2	3
1		s6		acc			
2		r2	s7	r2			
3		r4	r4	r4			
4		r6	r6	r6			
5	s4	s5					8
6	s4	s5				9	3
7	s4	s5					10
8		r5	r5	r5			
9		r1	s7	r1			
10		r3	r3	r3			

shift s': push(a); push(s'); next(ip);
 reduce by $A \rightarrow \beta$:
 pop($2 * |\beta|$); s' := top^;
 push(A);
 push(goto(s', A));
 write($A \rightarrow \beta$);

栈	剩余输入	动作
#0	id--id*id#	s4
#0id4	--id*id#	r6($F \rightarrow id$)
#0F3	--id*id#	r4($T \rightarrow F$)
#0T2	--id*id#	r2($E \rightarrow T$)
#0E1	--id*id#	s6
#0E1-6	-id*id#	s5
#0E1-6-5	id*id#	s4
#0E1-6-5id4	*id#	r6($F \rightarrow id$)
#0E1-6-5F8	*id#	r5($F \rightarrow -F$)
#0E1-6F3	*id#	r4($T \rightarrow F$)
#0E1-6T9	*id#	s7
#0E1-6T9*7	id#	s4
#0E1-6T9*7id4	#	r6($F \rightarrow id$)
#0E1-6T9*7F10	#	r3($T \rightarrow T * F$)
#0E1-6T9	#	r1($E \rightarrow E - T$)
#0E1	#	acc



3.5 自下而上语法分析



1. DFA的每个状态识别的活前缀不同;

2. 对语法结构正确的输入序列进行分析的任意时刻, 分析器处于DFA的某状态 i 。此时, 状态 i 识别的活前缀出现在栈中, 状态 i 位于栈顶。

语法分析器根据状态 i 中包含的LR(0)项目指导下一步分析

3. 状态 i 中出现的LR(0)项目对状态 i 能够识别的活前缀有效。



3.5 自下而上语法分析

2.3 有效项目

定义 若存在最右推导 $S' \xRightarrow{*} \alpha A \omega \Rightarrow \alpha \beta_1 \beta_2 \omega$, 则称项目 $A \rightarrow \beta_1 \beta_2$ 对活前缀 $\alpha \beta_1$ 有效。

项目 $A \rightarrow \beta_1 \beta_2$ 对活前缀 $\alpha \beta_1$ 有效, 作用是:

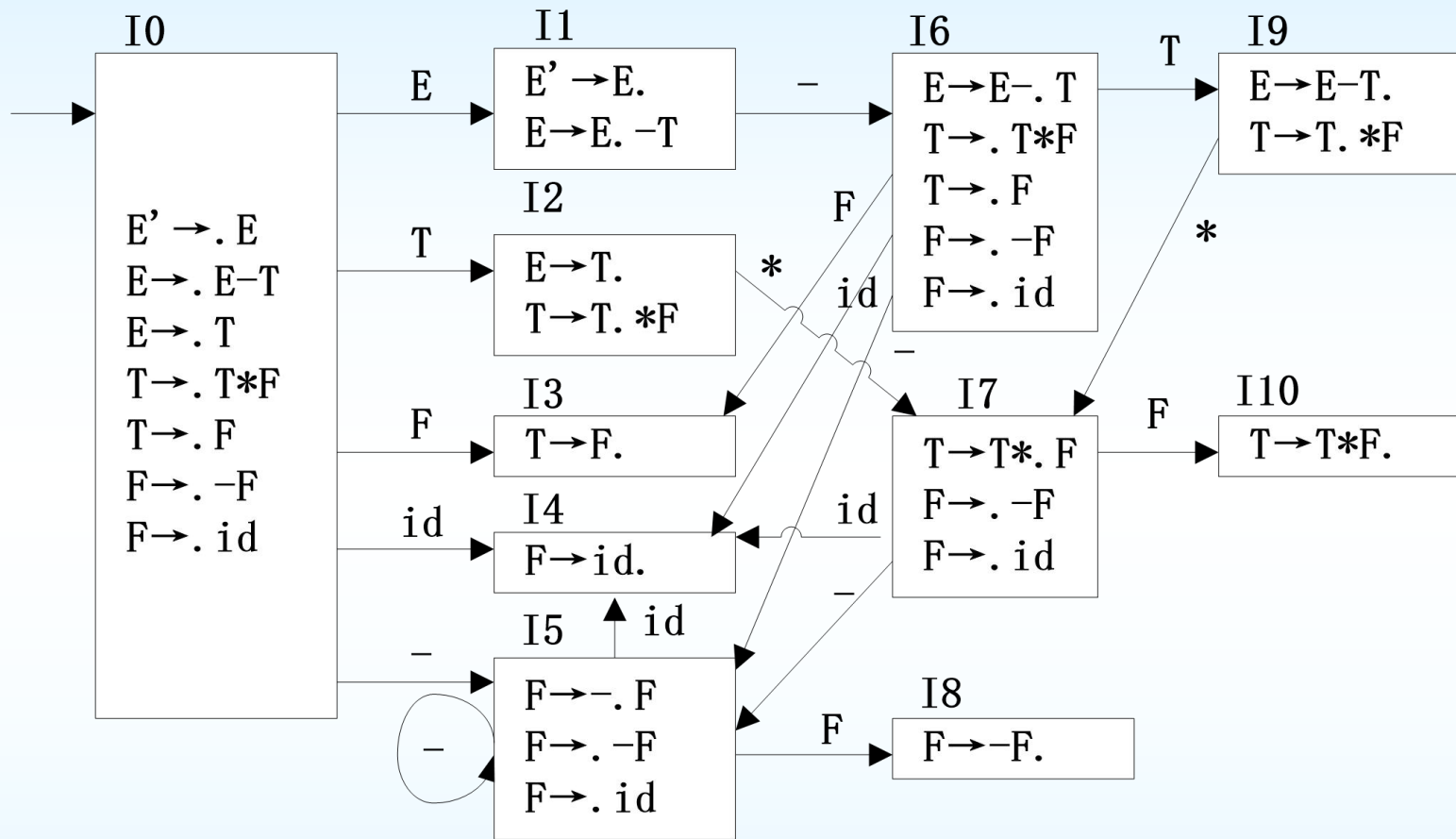
在当前活前缀 $\alpha \beta_1$ 的情况下, 该项目可指导下一步分析动作 ($\alpha A \omega \Rightarrow \alpha \beta_1 \beta_2 \omega$)。

活前缀与项目的关系:

- ※ 一个项目可能对若干个活前缀有效: 项目 $A \rightarrow \beta_1 \beta_2$ 对所有从初态出发可以到达此项目的路径上的标记均有效。
- ※ 若干个项目可能对同一个活前缀有效: 项目集中的所有项目对同一活前缀均有效。



3.5 自下而上语法分析



I5: $F \rightarrow - \cdot F$ 对活前缀 $T*-$ 、 $E--$ 和 $-$ 都有效。

I2, I9: $T \rightarrow T \cdot * F$ 对 T 、 $E-T$ 均有效。

I5: $F \rightarrow - \cdot F$ 、 $F \rightarrow \cdot - F$ 和 $F \rightarrow \cdot id$ 对 $T*-$ 均有效。



3.5 自下而上语法分析

综合可知：同一项目集中的所有项目，对此项目集的所有活前缀均有效。即项目集中的每个项目均有同等权利指导下一步动作。

有效项目的意义是：

① 到目前为止语法分析是正确的；

② **指导下一步的分析**：

$A \rightarrow \alpha.a\beta$ （可移进项）：若当前剩余输入为终结符 a ，则移进终结符 a 。

$B \rightarrow \beta.$ （可归约项）：按产生式 $B \rightarrow \beta$ 归约。



3.5 自下而上语法分析

2.4 LR(0)分析表的构造

定义 (LR(0)文法) 假若一个文法G的拓广文法G'的识别活前缀自动机中的每个状态（项目集）不存在下述情况：

- (1) 既含移进项目又含归约项目（**移进-归约冲突**）；
- (2) 含有多个归约项目（**归约-归约冲突**），

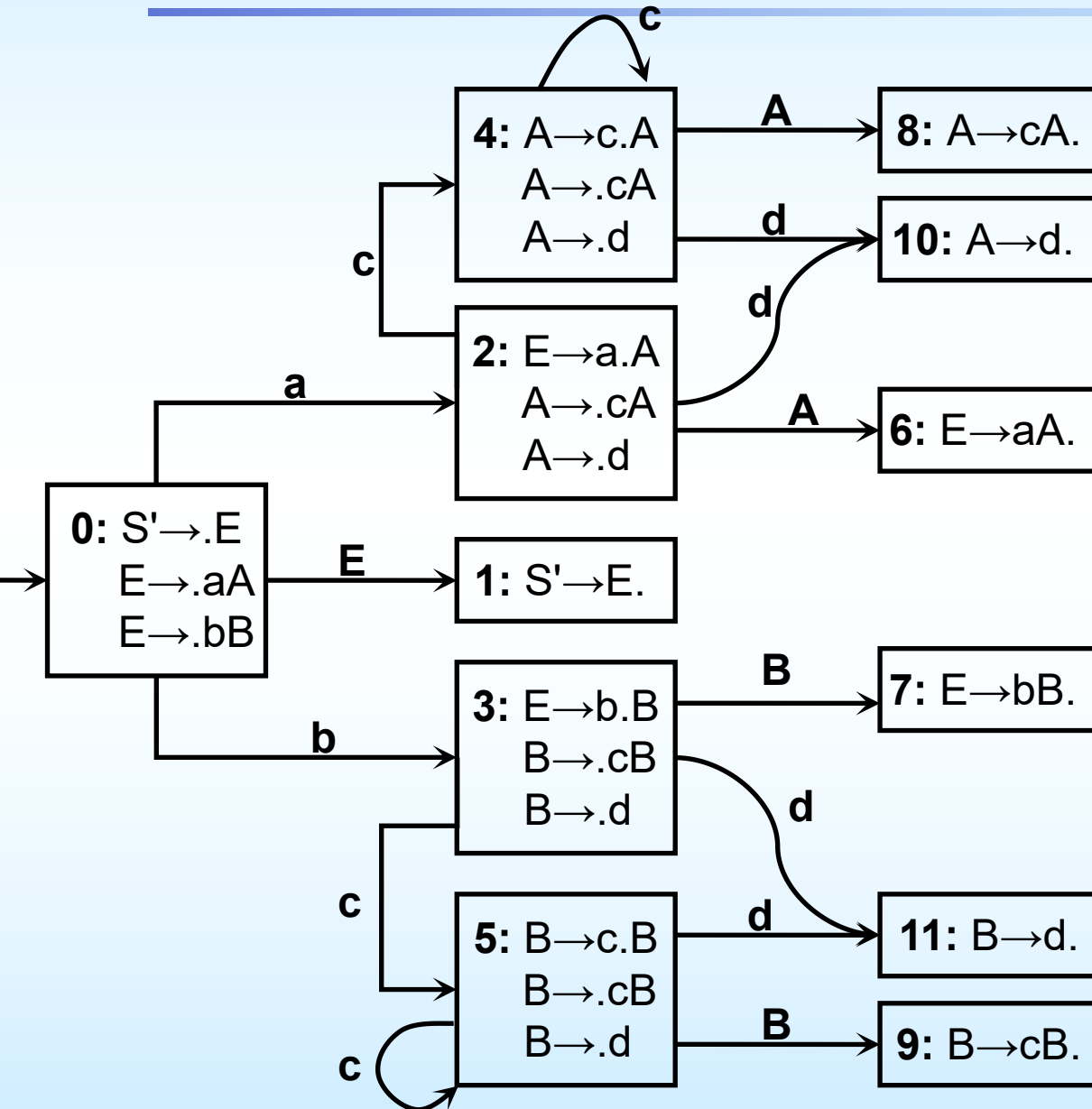
则称G是一个LR(0)文法。

LR(0)文法规范族的每个项目集不包含任何冲突项目。

对于**LR(0)文法**，可**直接通过**文法的项目集规范族(**识别活前缀的DFA**)构造LR分析表。



3.5 自下而上语法分析



$G: E \rightarrow aA \mid bB$

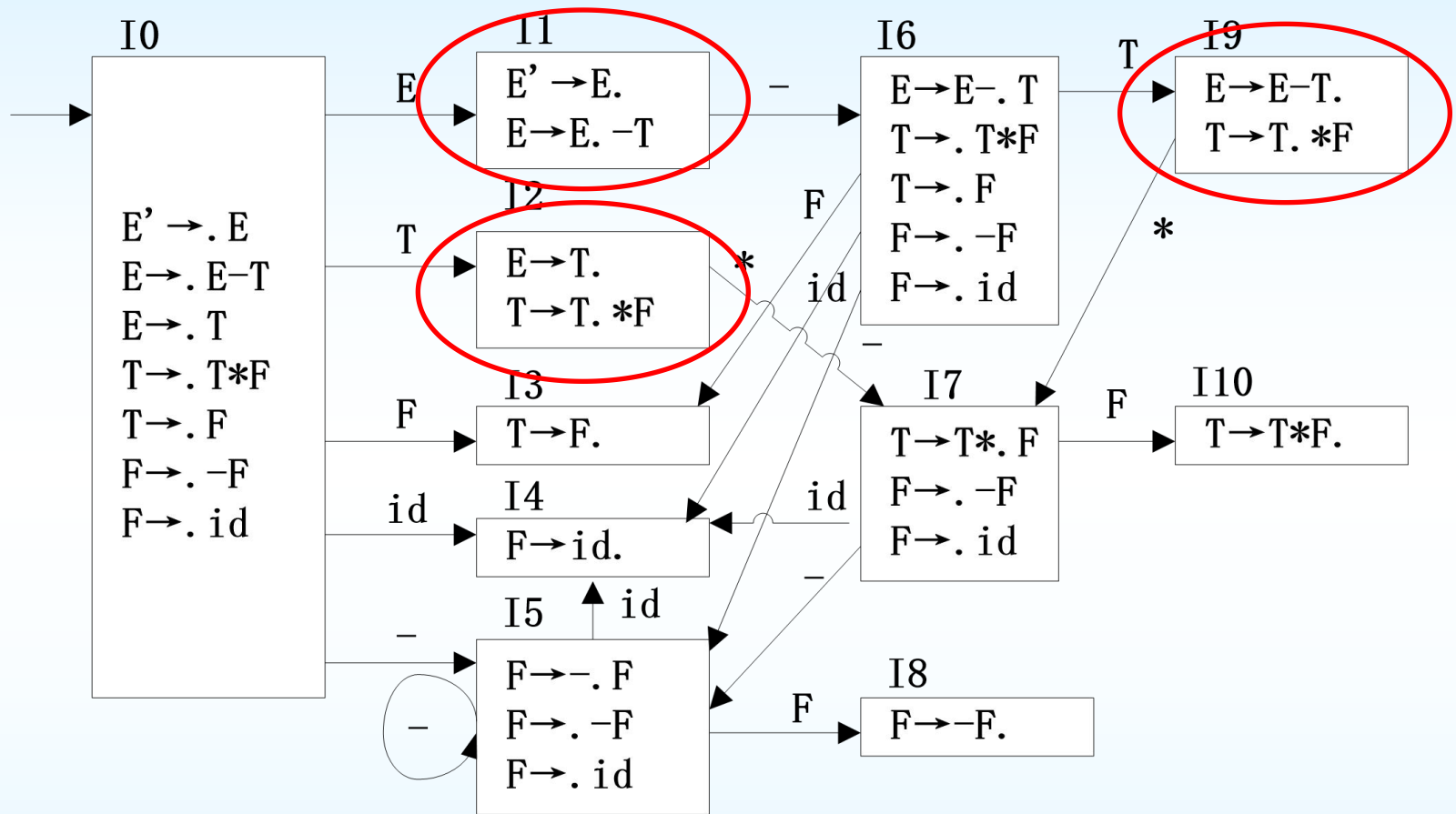
$A \rightarrow cA \mid d$

$B \rightarrow cB \mid d$

因为不含冲突， G 是
LR(0)文法。



3.5 自下而上语法分析



$G: E \rightarrow E-T | T \quad T \rightarrow T * F | F \quad F \rightarrow -F | id$

I1、I2、I9存在冲突，故G不是LR(0)文法。



3.5 自下而上语法分析

2.4 LR(0)分析表的构造

LR(0)分析表的构造方法:

为产生式编号（从1开始，不含拓广文法开始符号产生式）。假定 $C=\{I_0, I_1, \dots, I_n\}$ 。令每个项目集 I_k 的下标 k 作为分析器的状态，令包含项目 $S' \rightarrow \cdot S$ 的集合 I_k 的下标 k 为分析器的初态。ACTION子表和GOTO子表构造如下：

- (1) 若项目 $A \rightarrow \alpha \cdot a\beta$ 属于 I_k 且 $GO(I_k, a)=I_j$ ， a 为终结符，则置ACTION[k, a]为“将(j,a)移进栈”，简记为“sj”；
- (2) 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k ，则对任何终结符 a (或结束符#)，置ACTION[k, a]为“用产生式 $A \rightarrow \alpha$ 进行归约”，简记为“rj”（ $A \rightarrow \alpha$ 为文法 G' 的第 j 个产生式）；
- (3) 若项目 $S' \rightarrow S \cdot$ 属于 I_k ，则置ACTION[k, #]为“接受”，简记为“acc”；
- (4) 若 $GO(I_k, A)=I_j$ ， A 为非终结符，则置GOTO[k, A]=j；
- (5) 分析表中凡不能用规则(1)~(4)填入信息的空白格均置为“报错标志”。

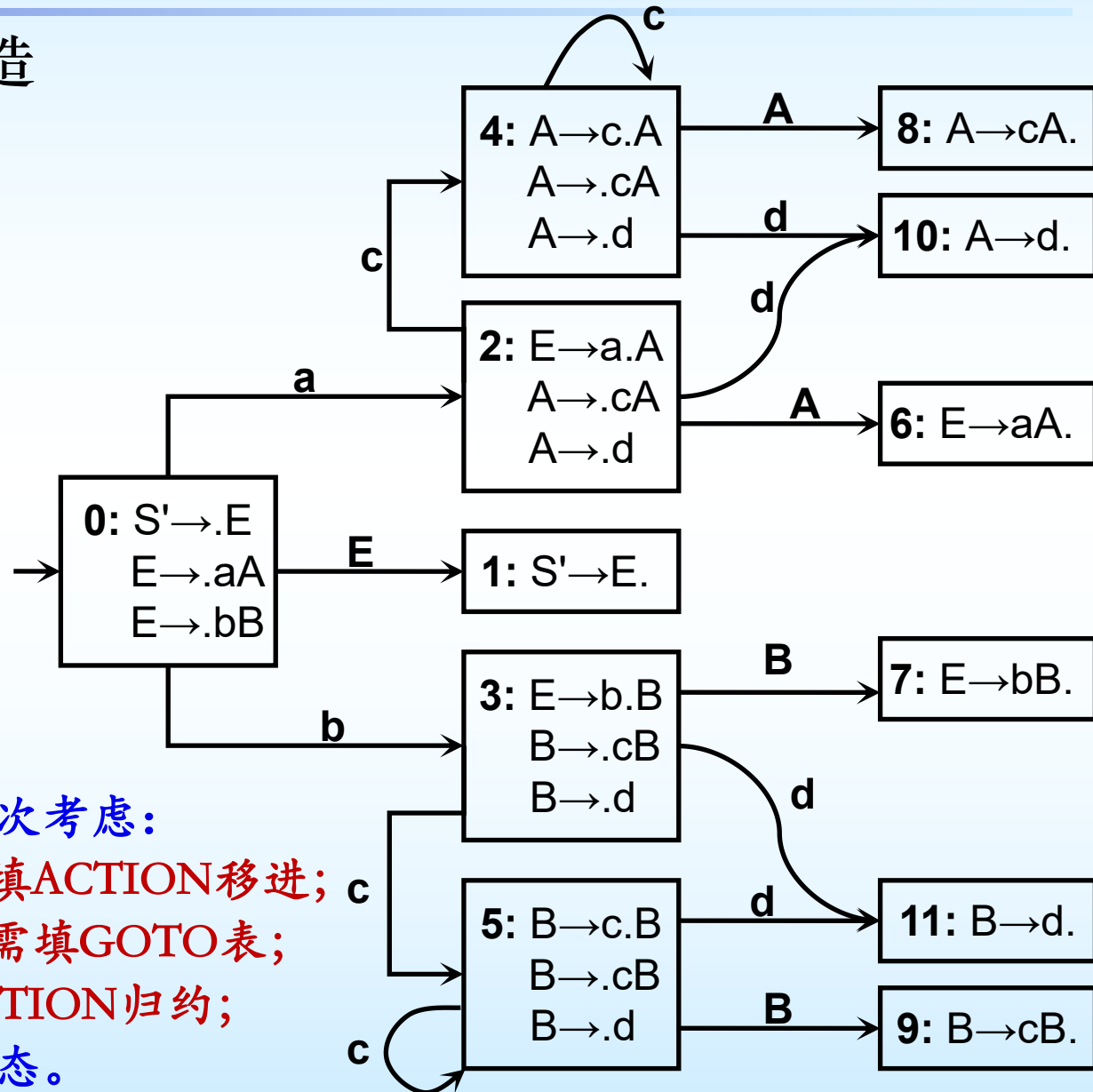
因为LR(0)文法规范族的每个项目集不含冲突项目，因此分析表中不含多重定义条目。



3.5 自下而上语法分析

2.4 LR(0)分析表的构造

- (0) $S' \rightarrow E$
- (1) $E \rightarrow aA$
- (2) $E \rightarrow bB$
- (3) $A \rightarrow cA$
- (4) $A \rightarrow d$
- (5) $B \rightarrow cB$
- (6) $B \rightarrow d$



每个状态依次考虑;

每个状态中的每个项目依次考虑:

圆点后是终结符, 需填ACTION移进;

圆点后是非终结符, 需填GOTO表;

圆点后为空, 需填ACTION归约;

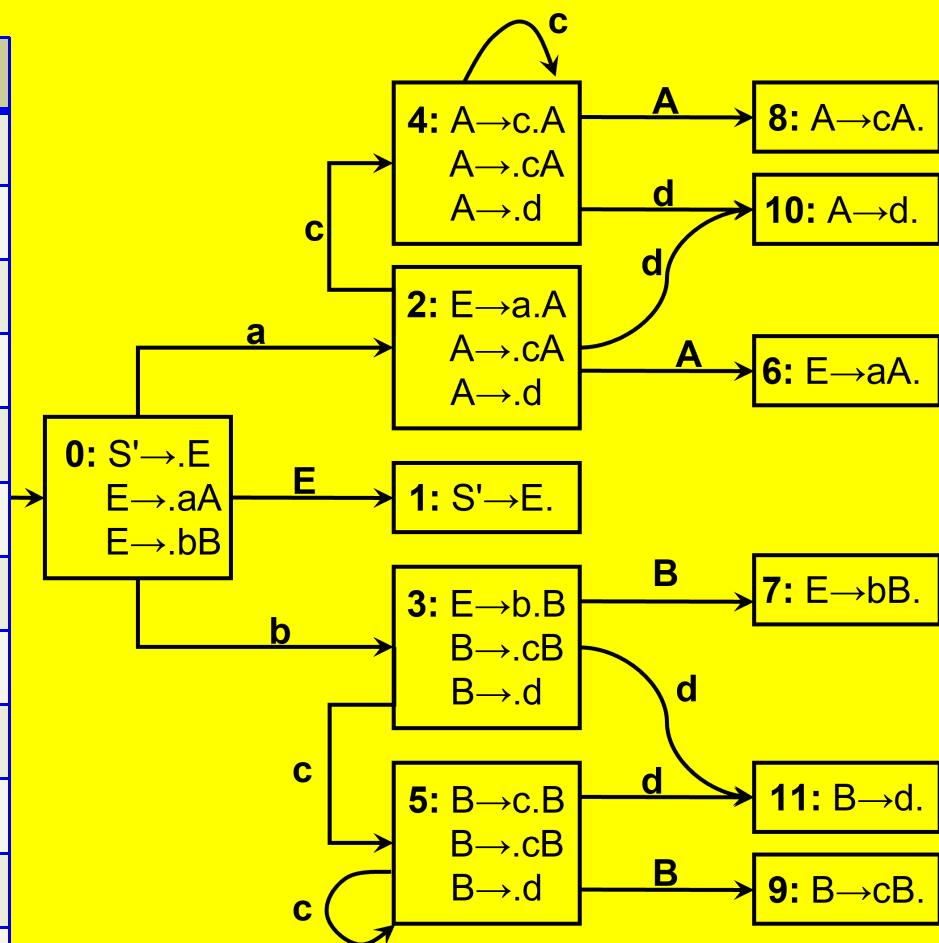
指明分析器的初态和接受态。

2.4 LR(0)分析表的构造

(0) $S' \rightarrow E$ (1) $E \rightarrow aA$ (2) $E \rightarrow bB$

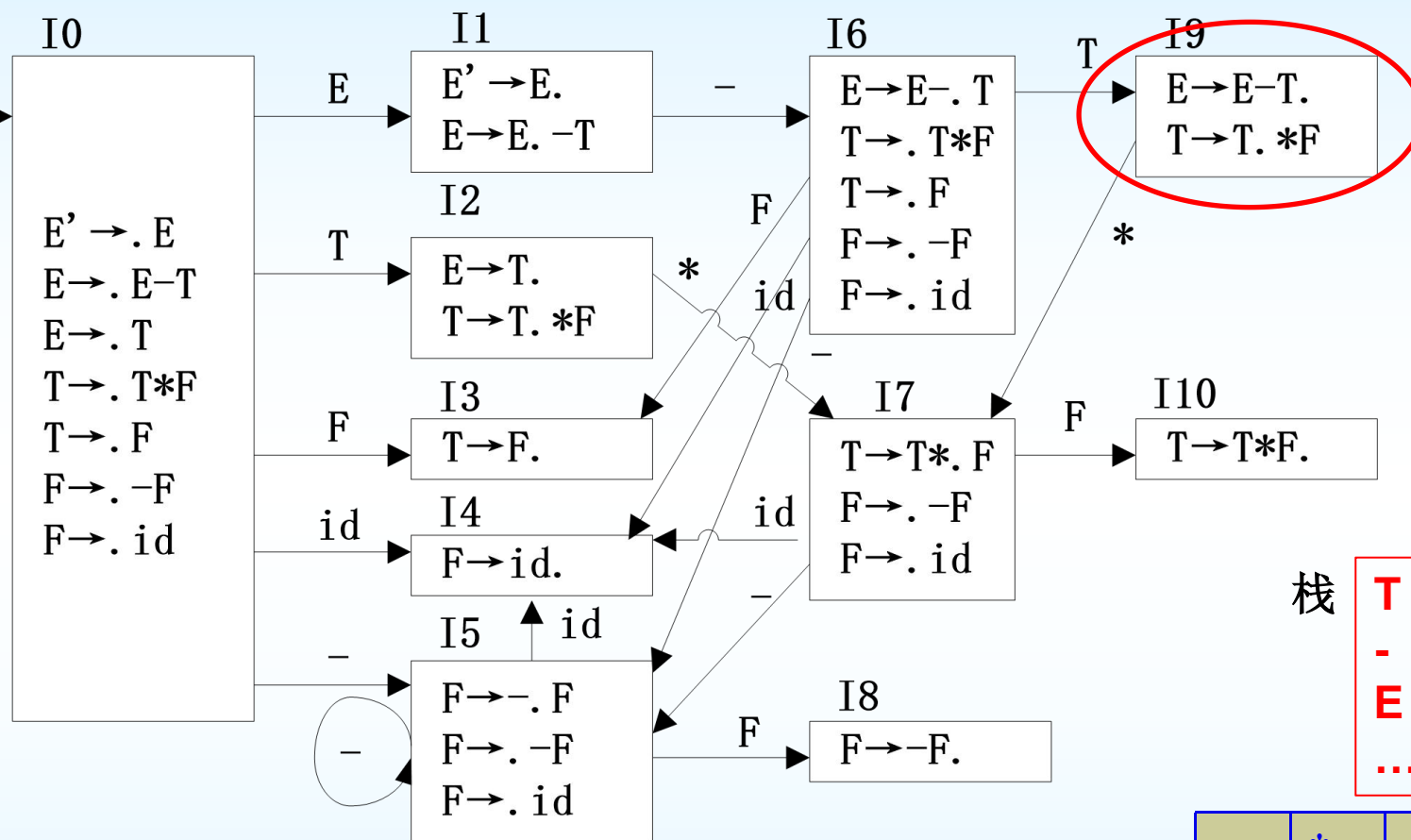
(3) $A \rightarrow cA$ (4) $A \rightarrow d$ (5) $B \rightarrow cB$ (6) $B \rightarrow d$

	a	b	c	d	#	E	A	B
0	s2	s3				1		
1					acc			
2			s4	s10			6	
3			s5	s11				7
4			s4	s10			8	
5			s5	s11				9
6	r1	r1	r1	r1	r1			
7	r2	r2	r2	r2	r2			
8	r3	r3	r3	r3	r3			
9	r5	r5	r5	r5	r5			
10	r4	r4	r4	r4	r4			
11	r6	r6	r6	r6	r6			





3.5 自下而上语法分析



剩余输入

*

栈

T
-
E
...

$G: E \rightarrow E-T|T \quad T \rightarrow T*F|F \quad F \rightarrow -F|id$
I1、I2、I9存在冲突，故G不是LR(0)文法。

	*			
9	s7, r1			



3.5 自下而上语法分析

3. SLR(1)分析表的构造(解决冲突的简单方法): 根据识别活前缀DFA和简单向前看一个终结符构造SLR分析表 (书3.3.8节)

问题: 假定LR(0)规范族的一个项目集I中含有m个移进项目:

$$A_1 \rightarrow \alpha_1 \cdot a_1 \beta_1, A_2 \rightarrow \alpha_2 \cdot a_2 \beta_2, \dots, A_m \rightarrow \alpha_m \cdot a_m \beta_m$$

同时含有n个归约项目:

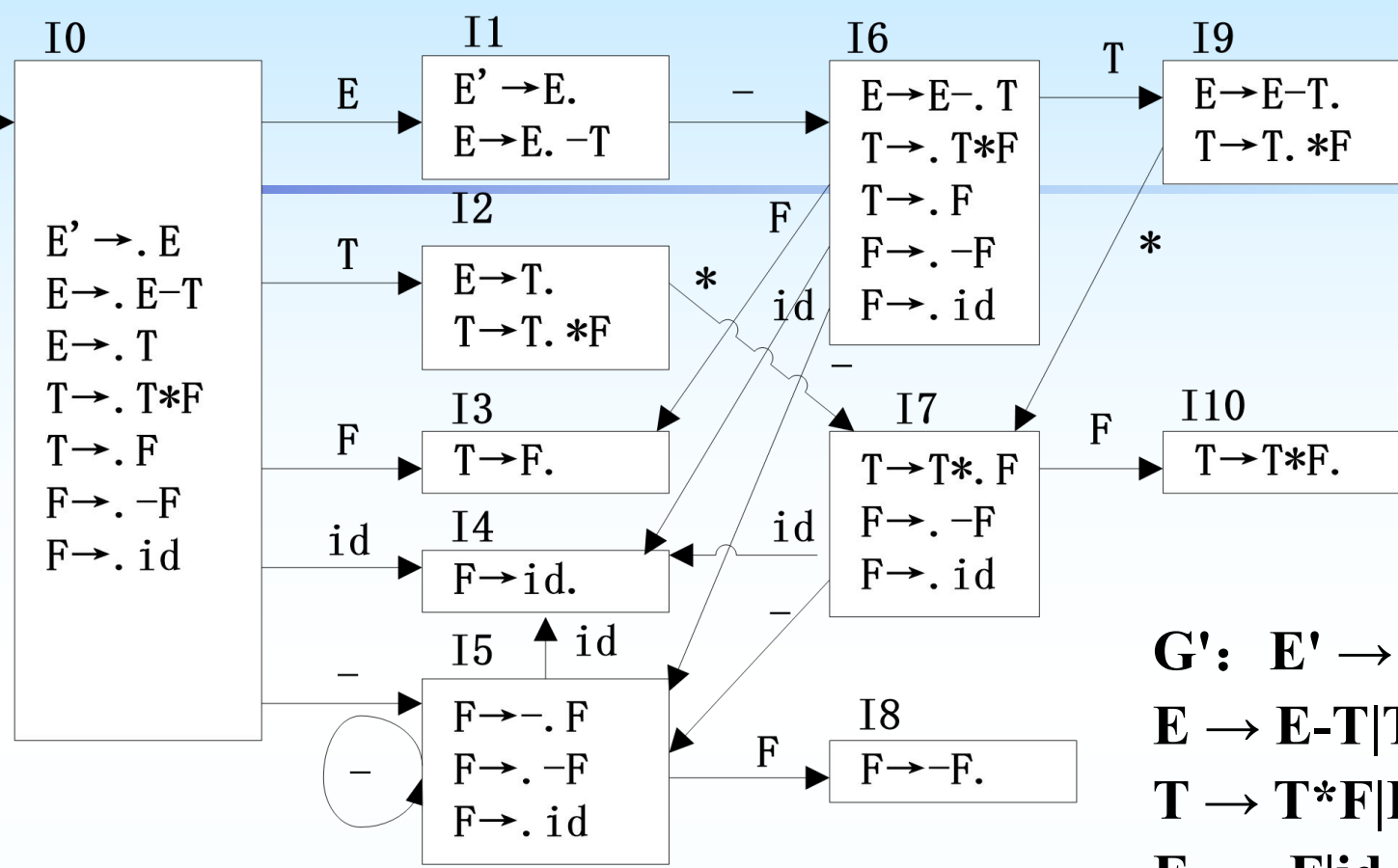
$$B_1 \rightarrow \alpha_1., B_2 \rightarrow \alpha_2., \dots, B_n \rightarrow \alpha_n.$$

如何解决移进-归约、归约-归约冲突?

如果集合 $\{a_1, \dots, a_m\}, FOLLOW(B_1), \dots, FOLLOW(B_n)$ 两两不相交, 则隐含在I中的动作冲突可通过检查当前剩余输入符号a属于上述n+1个集合中的哪个集合来解决,

- (1) 若a属于 $\{a_1, \dots, a_m\}$, 则移进;
- (2) 若a属于 $FOLLOW(B_i)$, 则用产生式 $B_i \rightarrow \alpha_i$ 进行归约;
- (3) 对(1)、(2)项以外的情况, 报错。

称这种冲突性动作的解决办法叫做SLR(1)解决办法。



$G': E' \rightarrow E$

$E \rightarrow E-T|T$

$T \rightarrow T*F|F$

$F \rightarrow -F|id$

I1、I2、I9，均有移进/归约冲突。

它不是LR(0)的，但是SLR(1)的，因为：

I1: $\{-\} \cap FOLLOW(E') = \Phi$

I2、I9: $\{*\} \cap FOLLOW(E) = \Phi$

$FOLLOW(E') = \{\#\}$

$FOLLOW(E) = \{-, \#\}$

$FOLLOW(T) = \{*, -, \#\}$

$FOLLOW(F) = \{*, -, \#\}$



④ SLR分析表的构造

对任给的文法G,

- 1、将G拓广为G';
- 2、为G'构造LR(0)项目集规范族C和识别活前缀的自动机的状态转换函数GO;
- 3、根据C和GO按照SLR(1)解决办法构造G'的分析表:

为产生式编号。假定 $C=\{I_0, I_1, \dots, I_n\}$ 。令每个项目集 I_k 的下标k为分析器的一个状态,令含项目 $S' \rightarrow \cdot S$ 的 I_k 的下标k为初态,则子表ACTION和GOTO按如下方法构造:

- (1) 若项目 $A \rightarrow \alpha \cdot a\beta$ 属于 I_k 且 $GO(I_k, a)=I_j$, a为终结符,则置ACTION[k, a]为“将(j,a)移进栈”,简记为“sj”;
- (2) 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k , 则对**属于FOLLOW(A)的符号a**, 置ACTION[k, a]为“用产生式 $A \rightarrow \alpha$ 进行归约”, 简记为“rj” ($A \rightarrow \alpha$ 为文法G'的第j个产生式);
- (3) 若项目 $S' \rightarrow S \cdot$ 属于 I_k , 则置ACTION[k, #]为“接受”, 简记为“acc”;
- (4) 若 $GO(I_k, A)=I_j$, A为非终结符, 则置GOTO[k, A]=j;
- (5) 分析表中凡不能用规则(1)~(4)填入信息的空白格均置为“报错标志”。

按照以上方法, 如果构造的ACTION和GOTO表的每个入口都不含多重定义, 则称它为文法G的一张SLR(1)表。具有SLR(1)表的文法G称为一个SLR(1)文法。使用SLR(1)表的分析器叫做一个SLR(1)分析器。



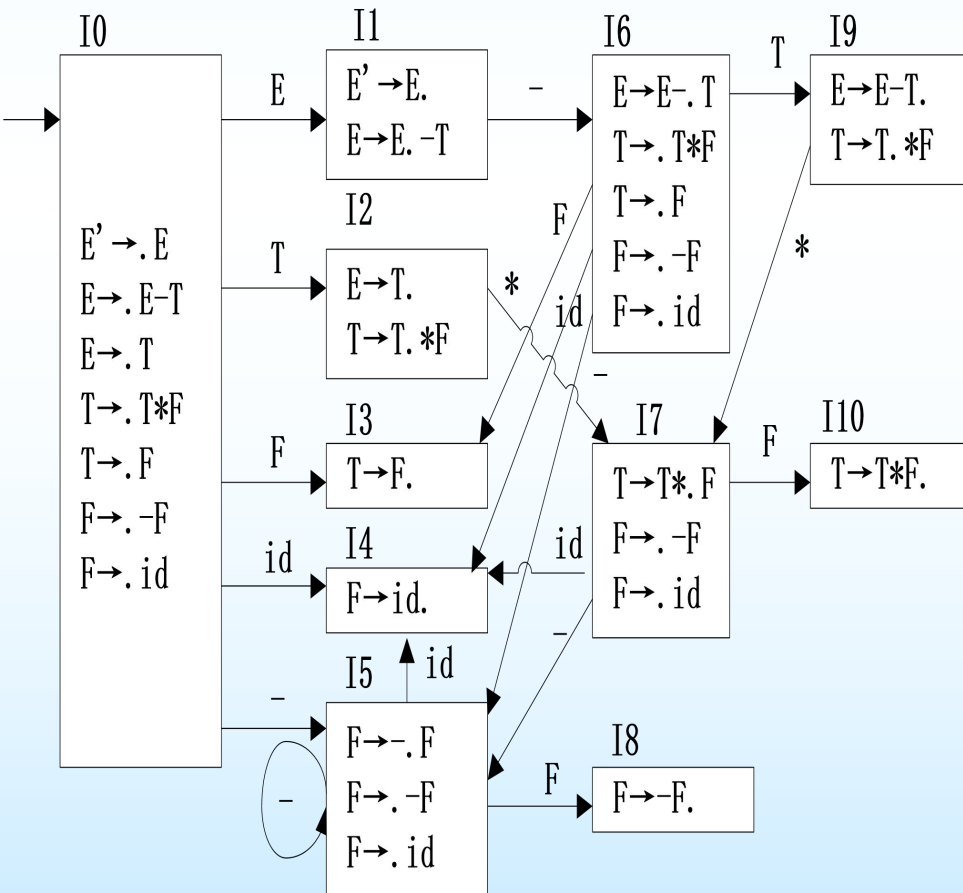
3.5 自下而上语法分析

每个状态转移 $Dtran[i,x]=j$: $action[i,x]:=Sj$; 或 $goto[i,x]:=j$;

每个状态 i 的可归约项 $A \rightarrow \alpha$: $action[i, \#]:=acc$; 或 $action[i,a]:=Rk$;

$FOLLOW(E') = \{\#\}$ $FOLLOW(E) = \{-, \#\}$

$FOLLOW(T) = FOLLOW(F) = \{*, -, \#\}$



	id	-	*	#	E	T	F
0	s4	s5			1	2	3
1		s6		acc			
2		r2	s7	r2			
3		r4	r4	r4			
4		r6	r6	r6			
5	s4	s5					8
6	s4	s5				9	3
7	s4	s5					10
8		r5	r5	r5			
9		r1	s7	r1			
10		r3	r3	r3			



3.5 自下而上语法分析

4. 非SLR(1)文法

① 二义文法不是SLR(1)文法

“悬空” else 文法:

$A \rightarrow S$

$S \rightarrow iCtSS' | a$

$S' \rightarrow eS | \varepsilon$

$C \rightarrow b$

$\text{FOLLOW}(A) = \{\#\}$

$\text{FOLLOW}(S) = \{\#, e\}$

$\text{FOLLOW}(S') = \{\#, e\}$

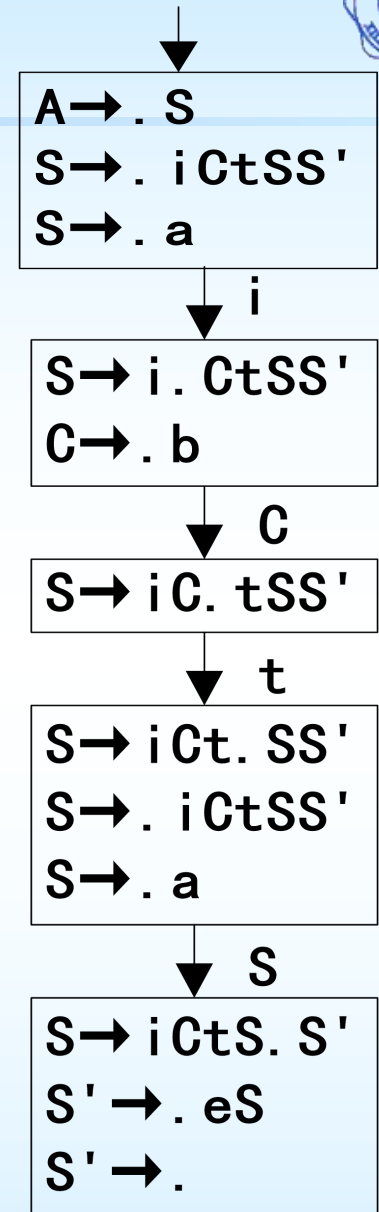
$\text{FOLLOW}(C) = \{t\}$

经过路径 $iCtS$ 达到的状态中存在

移进/归约冲突: $S' \rightarrow .eS$ $S' \rightarrow .$

因为 e 属于 $\text{FOLLOW}(S')$

所以冲突无法解决, 不是SLR(1)文法。

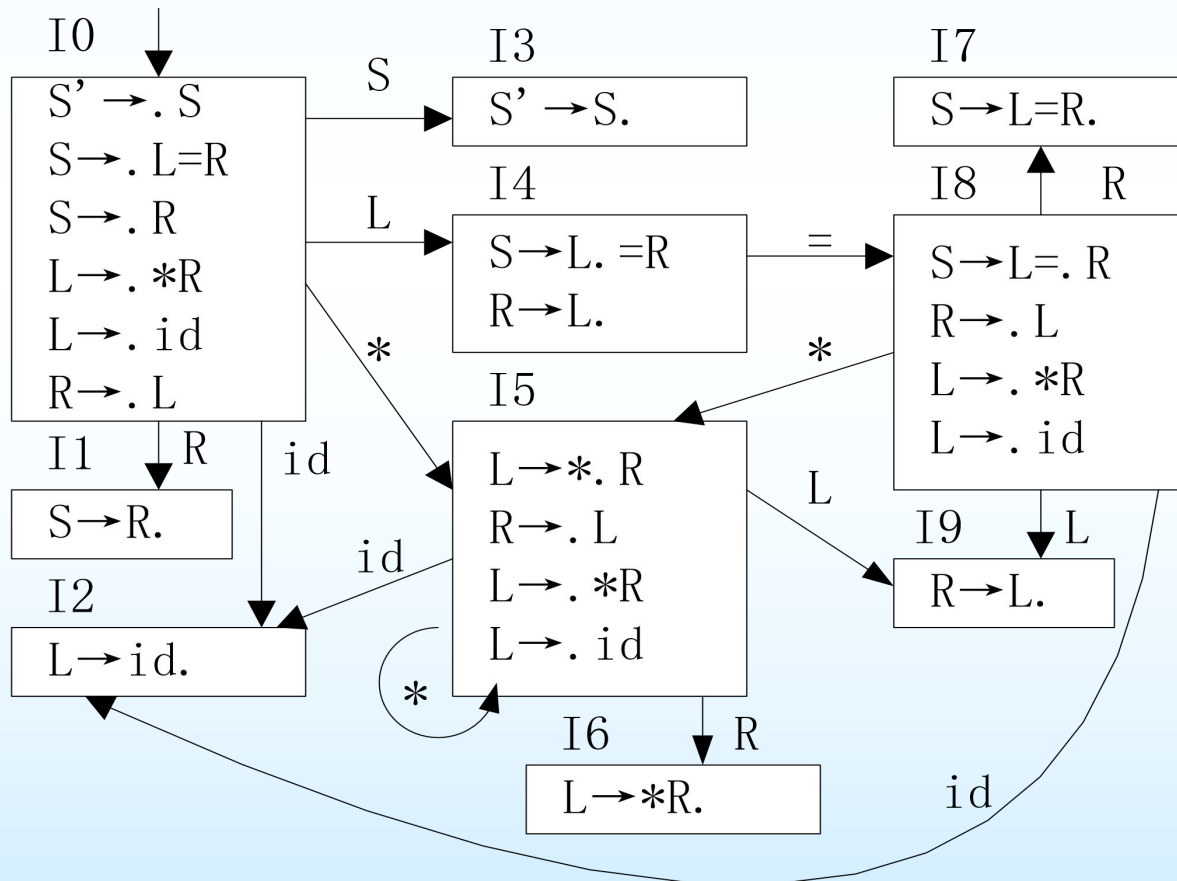


3.5 自下而上语法分析

② 不是二义文法的非SLR(1)文法

文法: $S \rightarrow L = R \mid R \quad L \rightarrow *R \mid \text{id} \quad R \rightarrow L$

识别活前缀的DFA:



**I4存在移进/归约冲突，不是LR(0)文法。
由于 = 属于 FOLLOW(R)故也不是SLR(1)文法。**

非二义文法可以增加向前看终结符个数解决冲突，如LL(2)或LR(2)；

二义文法无论向前看多少个终结符，也无法解决二义性。



3.5 自下而上语法分析

5. 基于LR分析的语法分析器生成器简介

利用YACC设计语法分析器，关键也是了解和掌握两点：

- ① YACC提供什么形式的产生式，如何运用它们设计语法分析器所需的文法；
- ② YACC提供什么样的机制支持语义动作的嵌入，如何运用这些机制进行语义处理，如算术表达式值的计算、构造所分析句子的语法树等。





3.6 本章小结

- ☞ 程序设计语言与文法
 1. 正规式与正规文法
 2. 上下文无关文法 $CFG = (N, T, P, S)$
 3. 文法分类：0型、1型、2型和3型
- ☞ 有关推导的基本概念
 1. 产生语言的基本方法—推导：句子与句型、直接推导与推导、最左推导与左句型
 2. 语法树与抽象语法树
 - 语法树记录推导过程并反映语言结构
 - 抽象语法树仅反映语言结构而忽略推导过程，树中没有非终结符



3.6 本章小结



自上而下分析

1. 分析方法：用推导的方法从上到下构造语法树，谋求与输入序列的匹配，是一种试探的方法；
2. 对文法的要求：为避免回溯与无穷递归，要求文法没有公共左因子和左递归；
3. 递归下降子程序：每个非终结符是一个子程序
4. 预测分析
 - ✱ 下推自动机工作模式：格局与格局的变换
 - ✱ 预测分析表：四个动作、分析表中的内容
 - ✱ 预测分析表的构造：FIRST集合与FOLLOW集合
5. LL(1)文法及其判别方法



👉 自下而上分析

1. 分析方法：用归约的方法从叶子到根构造语法树，谋求对输入序列的匹配
2. 基本概念：短语、直接短语、句柄、归约、规范归约、剪句柄
3. 基本方法：用移进-归约方法实现剪句柄
 - ① LR文法与LR分析：LR(0), SLR(1), LALR(1), LR(1)
 - ② SLR(1)分析
 - ✱ 格局与格局变换、改变格局的四个动作
 - ✱ 移进 - 归约分析表：动作表与转移表
 - ③ SLR(1)分析器的构造
 - ✱ 识别活前缀的DFA：LR(0)项目、项目集、项目集族、子集法
 - ✱ 识别活前缀：有效项目、可移进项、可归约项、冲突
 - ④ SLR(1)文法：简单向前看一个终结符